

HONEYWELL EMBEDDED ENGINEERING TEAMS — PA · IA · BA

AI Champions Programme

Module 02

GitHub Copilot Enterprise Features for Embedded Engineering

Turning a familiar autocomplete tool into a disciplined engineering partner — across C/C++ source, headers, build systems, and your existing engineering conventions.



DURATION

2 Hours · Module 2 of 12



FORMAT

Hands-On Lab + Guided Demo



DELIVERED FOR

Honeywell Embedded Engineering

How We'll Spend These Two Hours

Six connected moves — from a capability map to a measured comparison against solo work.



01

Capability Map Walkthrough

15 min

Chat, inline assist, explain, refactor, debug, test-gen, agent mode, reusable skills — one map, no guesswork later.



02

Generate, Explain & Refactor Live

20 min

On the sample C/C++ repo: generate a function, explain an unfamiliar module, refactor safely.



03

Debug a Real Defect

20 min

Hunt and fix an injected defect in the driver layer with Copilot chat + inline assist.



04

Generate Tests & Review a PR

20 min

Host-based test generation, plus a Copilot-assisted PR / code-review pass.



05

Agent Mode on a Repository Task

25 min

Hand Copilot a scoped, repository-aware task and supervise it end-to-end.



06

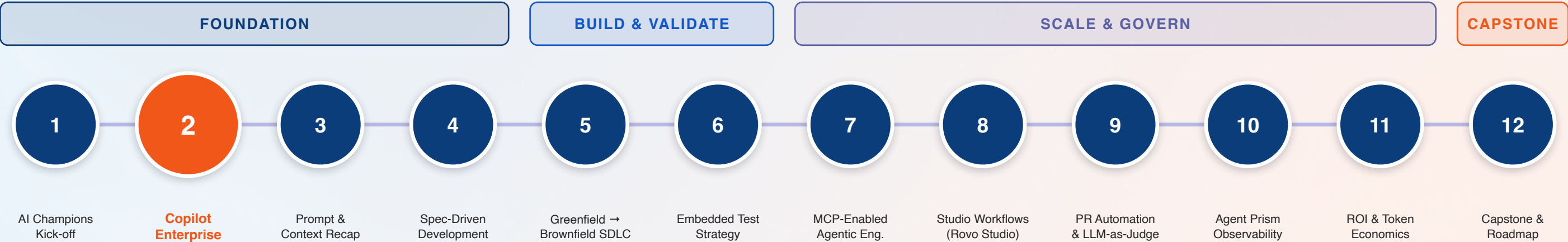
Manual vs AI-Assisted Comparison

20 min

Time, quality & confidence — measured against Module 01's cohort baseline.

Module 02 in the 12-Module Journey

Capability-building before discipline — Module 03 recaps prompting, Module 04 adds the specification layer.



Why this matters: Every later module assumes fluency with these Copilot capabilities. The context-engineering work in Module 03 and the SDD workflow in Module 04 are built directly on top of chat, inline assist, and agent mode — not around them.

GitHub Copilot Enterprise: The Capability Map

Thirteen capabilities, five clusters — the vocabulary we'll use for the rest of the programme.



Conversational & Inline Assistance

- Chat / Conversation
- Inline Code Assistance
- Code Explanation



Code Quality Assistance

- Refactoring
- Debugging
- Test Generation



Autonomous & Repository-Aware

- Agent Mode
- Repository-Aware Assistance
- Model / Task Selection



Workflow Integration

GitHub.com Workflows · Terminal / CLI Workflows · PR / Code-Review Assistance



Reuse & Governance

Custom / Reusable Instructions & Skills · Safe Enterprise Usage

Cohort ranking, pre-training survey: Code generation, architecture & design assistance, and debugging topped the list of capabilities engineers most want — exactly where today's hands-on time goes.

What the Calibration Survey Told Us About Copilot Usage Today

Real answers from the same 19 engineers — showing exactly how much headroom this module unlocks.

How Often Do You Use an AI Coding Assistant?

16

of 19

• Daily (6)

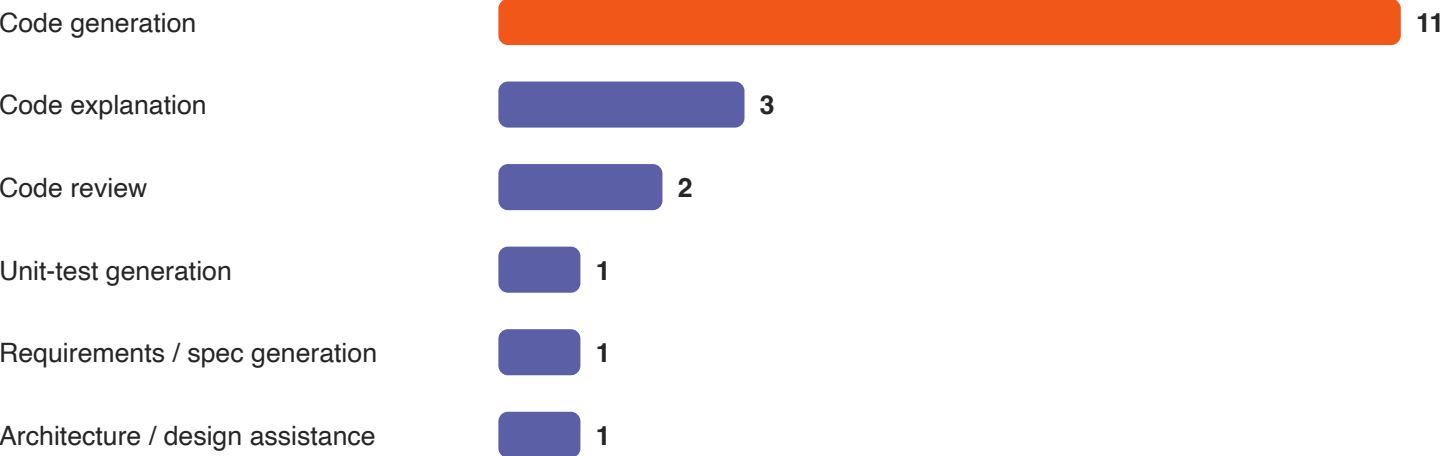
• Regularly (10)

• Occasionally (3)

16 of 19 use it daily or regularly. This cohort isn't new to Copilot — the gap is breadth of capability, not adoption.

How context is given today: 10 of 19 rely mainly on prompts alone; only 4 provide repository context up front — the exact gap Module 03 closes.

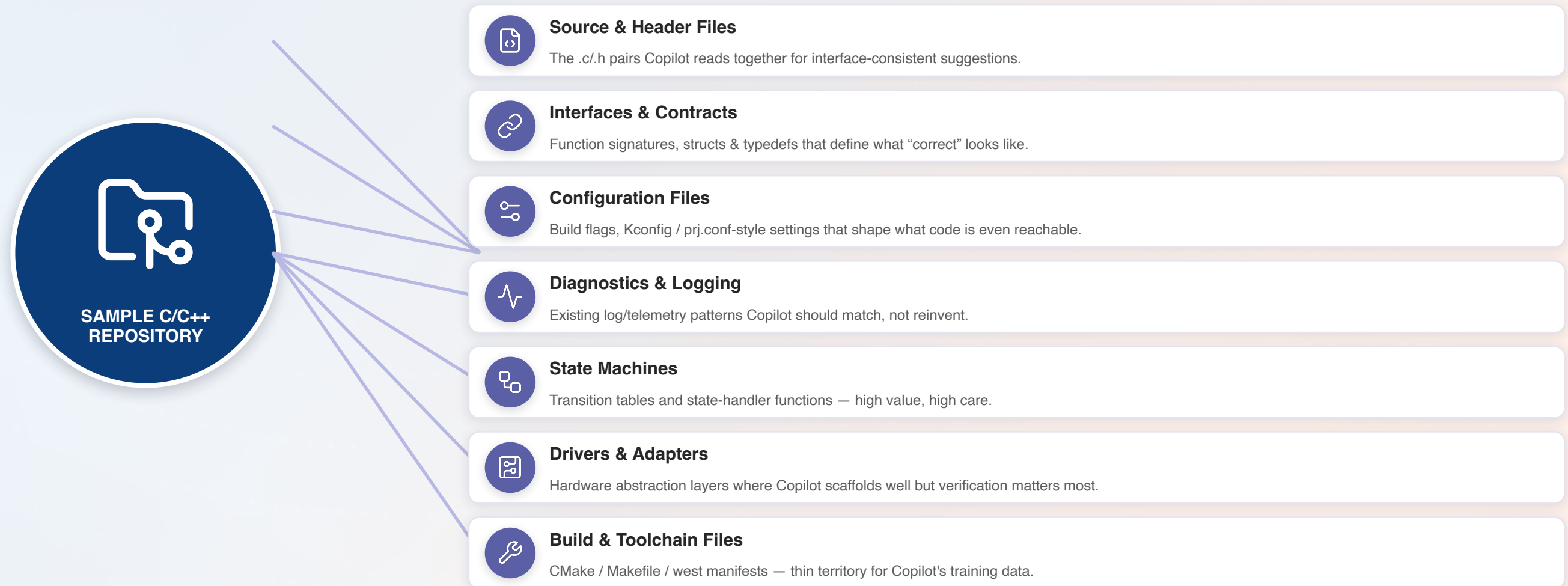
Capabilities Already Tried With an AI Assistant



The headroom this module unlocks: 11 of 19 have only tried code generation. Debugging, refactoring, agent mode & PR review are largely untouched — today changes that.

Where Copilot Touches Your Embedded Repository

The same seven surfaces recur across every hands-on exercise this week.



Chat, Inline Assist & Code Explanation in Practice

Three everyday moves — the ones you'll reach for in nearly every session.



Chat / Conversation

Ask before you touch anything

“Explain how the CAN driver's retry logic handles bus-off recovery before I modify it.” Copilot reasons over the repository, not just the open file.



Inline Code Assistance

Matches your coding style

Start a new diagnostic message formatter; Copilot completes it matching existing naming, error codes, and formatting conventions.



Code Explanation

Understand before you extend

Highlight an unfamiliar state-machine transition table and get a plain-language walkthrough of each branch and guard condition.

Refactor, Debug & Generate Tests

Where Copilot starts doing real engineering work — always under your review.



Refactoring

Restructure without regressing

Break a monolithic sensor-polling loop into named helper functions — same behaviour, readable structure, verified with existing tests.



Debugging

Root-cause with context

Feed Copilot a failing test plus stack trace; ask for likely root cause in the driver layer before you start printf-debugging.



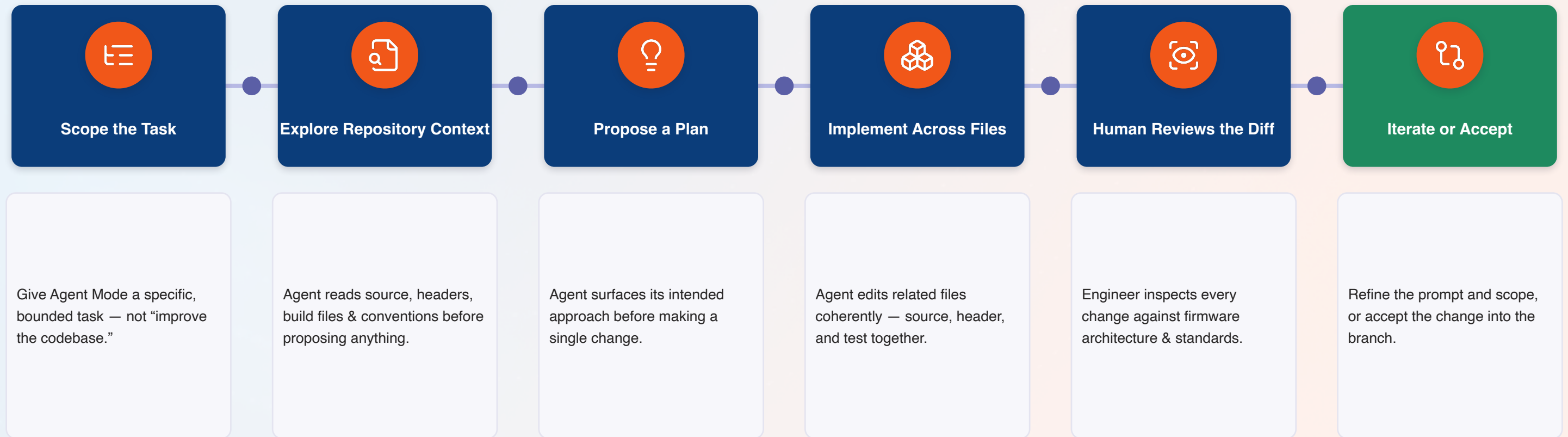
Test Generation

Cover the cases you'd skip

Generate host-based unit tests for a new HAL wrapper — including the boundary and negative cases easy to forget under deadline.

Agent Mode: Autonomous, Repository-Aware Assistance

Six steps, always ending with a human reviewing the diff — never a silent merge.



Rule of thumb: The narrower the scope you hand Agent Mode, the more trustworthy the diff. “Fix the retry counter in can_driver.c” beats “improve CAN reliability” every time.

Where Copilot Shines vs. Struggles in Embedded Code

Set your trust level by task type, not by tool reputation — the single most important slide in Module 02.

STRONG — WELL-REPRESENTED IN TRAINING DATA

- Boilerplate & scaffolding code — struct definitions, getters/setters, common patterns.
- Driver scaffolding & adapter patterns that follow conventional HAL structures.
- Documentation, comments & code explanation for readable, conventionally-written code.
- Test generation for well-defined, narrow interfaces with clear inputs/outputs.
- General C/C++ idioms & standard library usage — the “common knowledge” of the language.

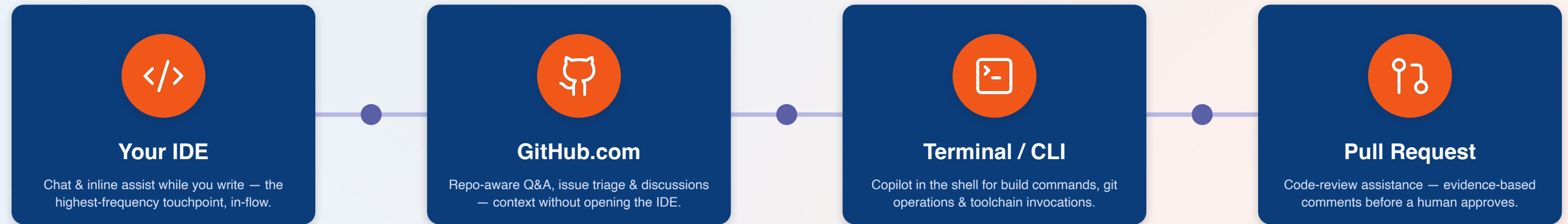


WEAK — THIN TRAINING DATA COVERAGE

- Closed-source vendor toolchains & proprietary APIs — no public documentation to learn from.
- RTOS internals (schedulers, memory managers) at implementation depth, not surface usage.
- AUTOSAR / CAPL and other narrow, licensed embedded standards with restricted public examples.
- Timing-critical & interrupt-sensitive logic where correctness depends on hardware behaviour.
- Undocumented hardware quirks, workarounds & tribal knowledge baked into brownfield code.

Workflow Integration: Where Copilot Meets Your Day

The same assistant, four different surfaces — each with a distinct job.



TODAY'S HANDS-ON PATH THROUGH ALL FOUR

Generate & refactor in the IDE → ask a repo-wide question on GitHub.com → run the test suite from the terminal with Copilot's help → open a PR and let Copilot draft the first review pass before a human signs off.

Reusable Instructions & Skills, and Safe Enterprise Usage

Making good practice repeatable across PA, IA & BA — and keeping it inside the guardrails.

CUSTOM & REUSABLE INSTRUCTIONS / SKILLS



Repository-level custom instructions

Encode coding standards, naming conventions & HAL patterns once — Copilot applies them on every suggestion.



Reusable skills & prompt packs

Package recurring tasks — driver scaffolding, test generation for a given interface style — as shareable assets.



Shared across PA / IA / BA

One GBE's well-tuned instructions become every GBE's starting point, curated by the AI Champion network.

SAFE ENTERPRISE USAGE



Approved repositories only

Copilot Enterprise sees only what your GitHub Enterprise Cloud tier grants — confirm scope before every session.



Never paste proprietary vendor docs elsewhere

Closed-source toolchain manuals & datasheets stay inside approved, governed tools — not public chat contexts.



PR review is never bypassed

Agent Mode and inline suggestions still go through the same human review & approval gate as any change.

Hands-On Lab Preview: Manual vs. AI-Assisted

Same six tasks, two ways — today's lab makes the difference measurable, not anecdotal.

TASK	MANUAL APPROACH	COPILOT-ASSISTED APPROACH
 Generate a new diagnostic function	Write it from scratch, by hand	Prompt Copilot with the existing formatter's style as context
 Explain an unfamiliar driver module	Read source + comments end-to-end	Ask Copilot Chat for a guided walkthrough, then verify
 Refactor the sensor-polling loop	Manually extract helper functions	Refactor with inline assist, diff against original behaviour
 Debug an injected defect	Trace manually with prints/debugger	Feed failing test + stack trace to Copilot Chat
 Generate unit tests for a HAL wrapper	Write boundary/negative cases by hand	Generate with Copilot, then audit for gaps
 Multi-file repository task	Not attempted solo in this timebox	Scope it for Agent Mode and supervise the diff

Module 02 Outcomes & What's Next

Everything below should be true before we recap prompting and add context engineering in Module 03.



All five capability clusters used hands-on: chat, code quality, autonomous mode, workflow, reuse



Debugged a live defect and generated host-based unit tests with Copilot's help



Can name where Copilot is strong (boilerplate, docs, scaffolding) vs. weak (RTOS internals, closed toolchains)



Understand reusable instructions/skills and the safe-usage guardrails for approved repositories



Comfortable generating, explaining, and refactoring inside a real C/C++ embedded repository



Ran Agent Mode on a scoped repository task and reviewed the resulting diff



Know all four workflow touchpoints: IDE, GitHub.com, terminal/CLI, and PR review



Captured a manual-vs-AI-assisted comparison to carry into the ROI conversation in Module 11



NEXT — MODULE 03: PROMPT ENGINEERING RECAP AND CONTEXT ENGINEERING

2 hours · From prompt-only context to structured repository & design context — comparing output quality, token usage and iteration count.

Questions & Discussion

Module 02 — GitHub Copilot Enterprise Features for Embedded Engineering